Школа анализа данных

Lecture 3

AI Agent Workflow
Multi-Agent Systems
Multimodality

Sofya Proskurina
Software Engineer at Yandex Lavka

Agenda

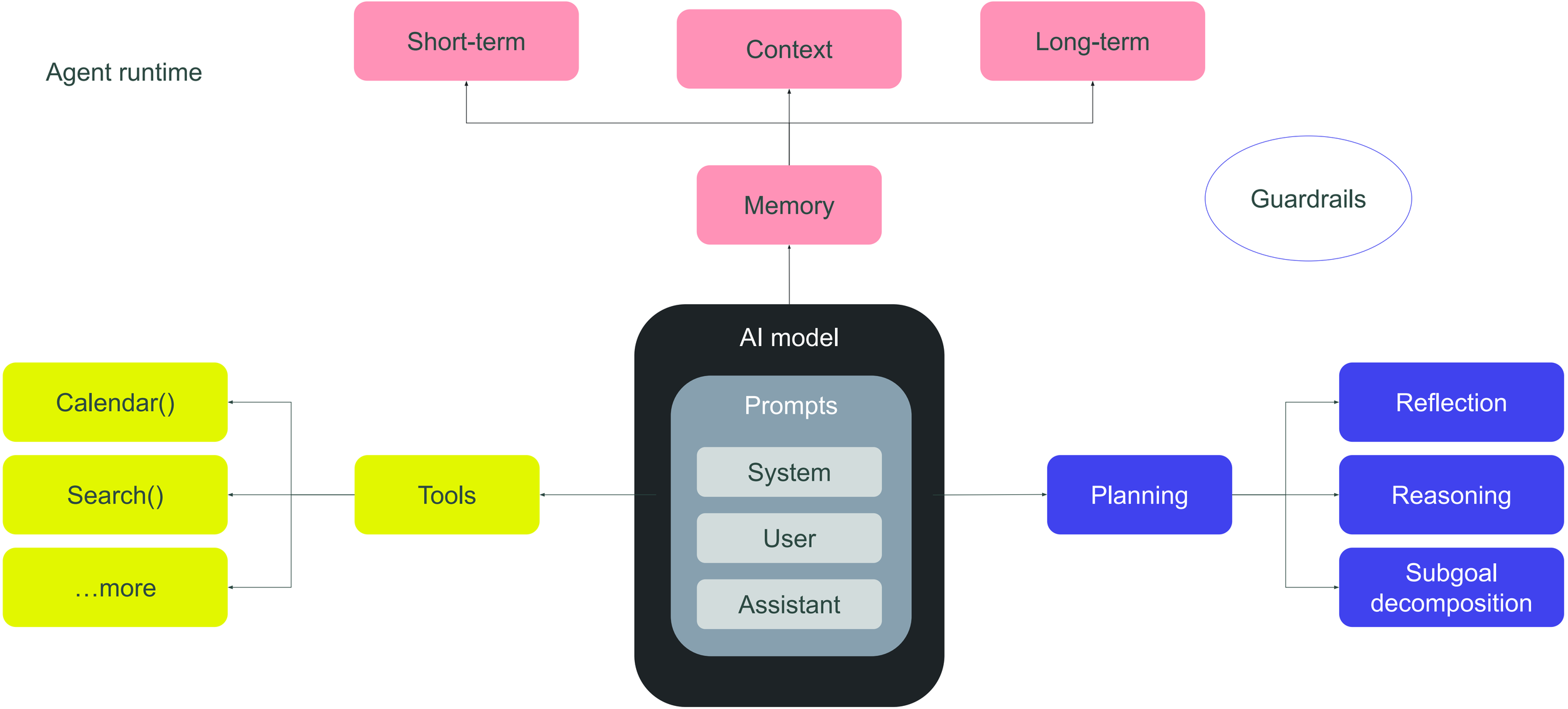
01 How Agents Think (TAO Loop)

02 Multi-Agent Systems (MAS)

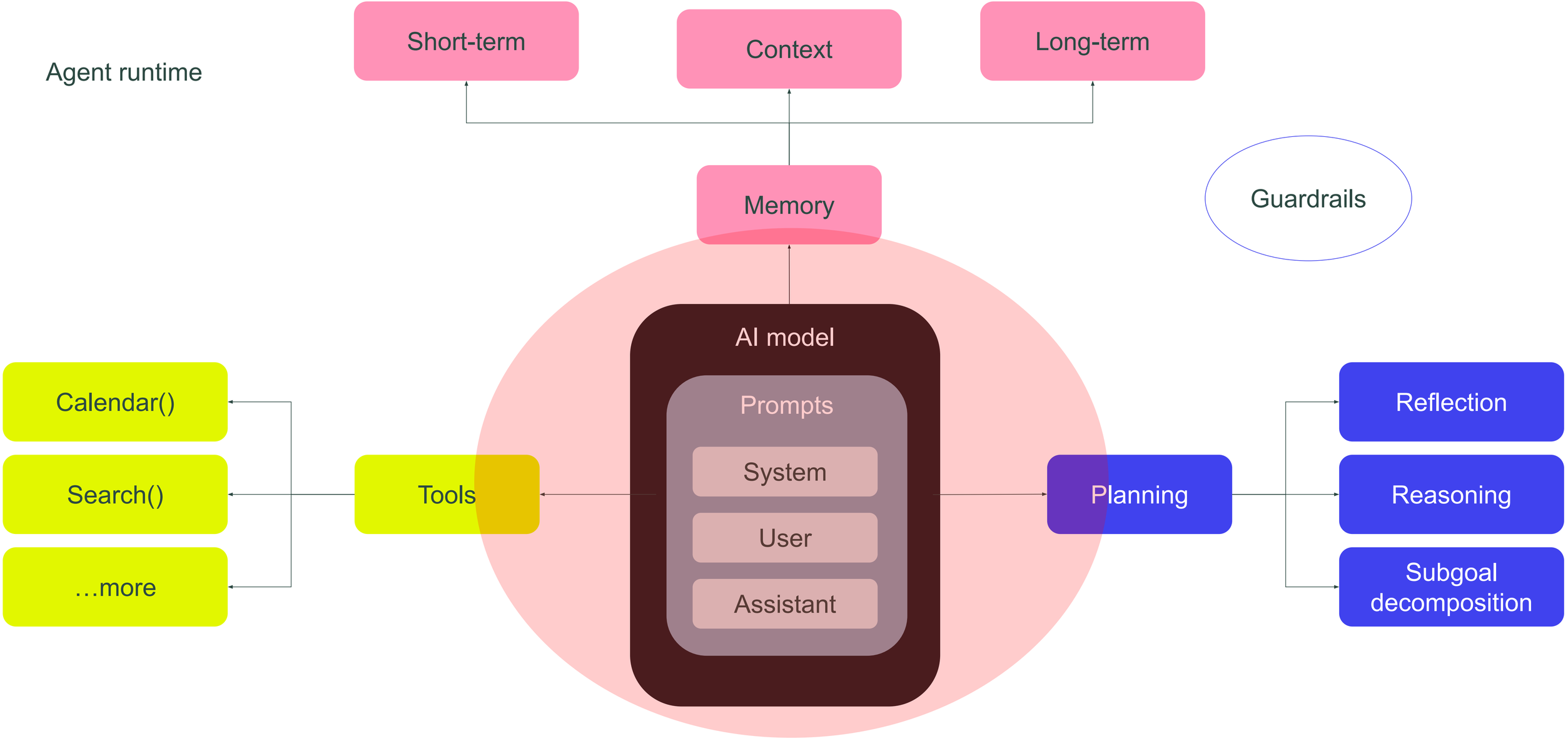
03 Multimodal Agents

04 Summary & Practice

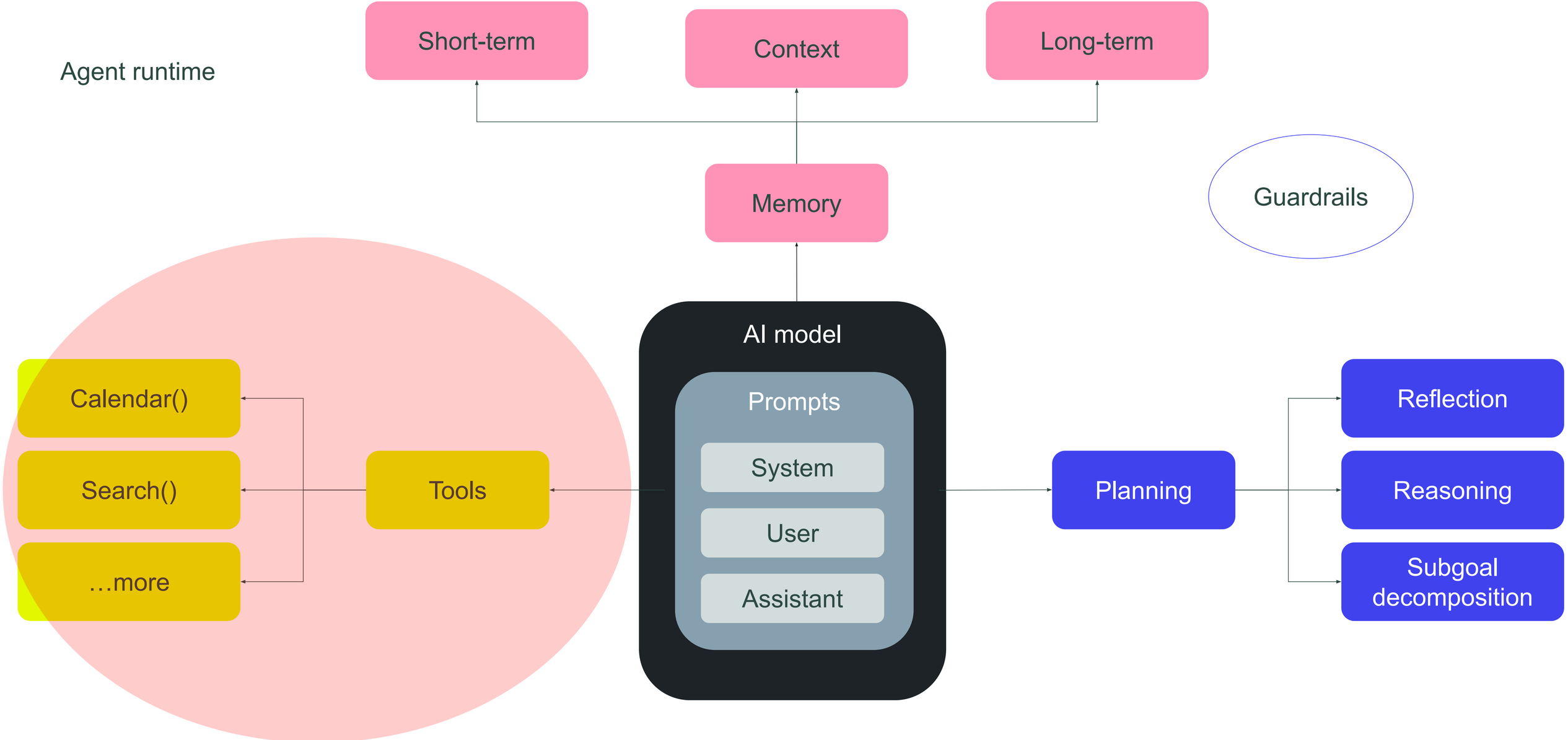
What We Already Know



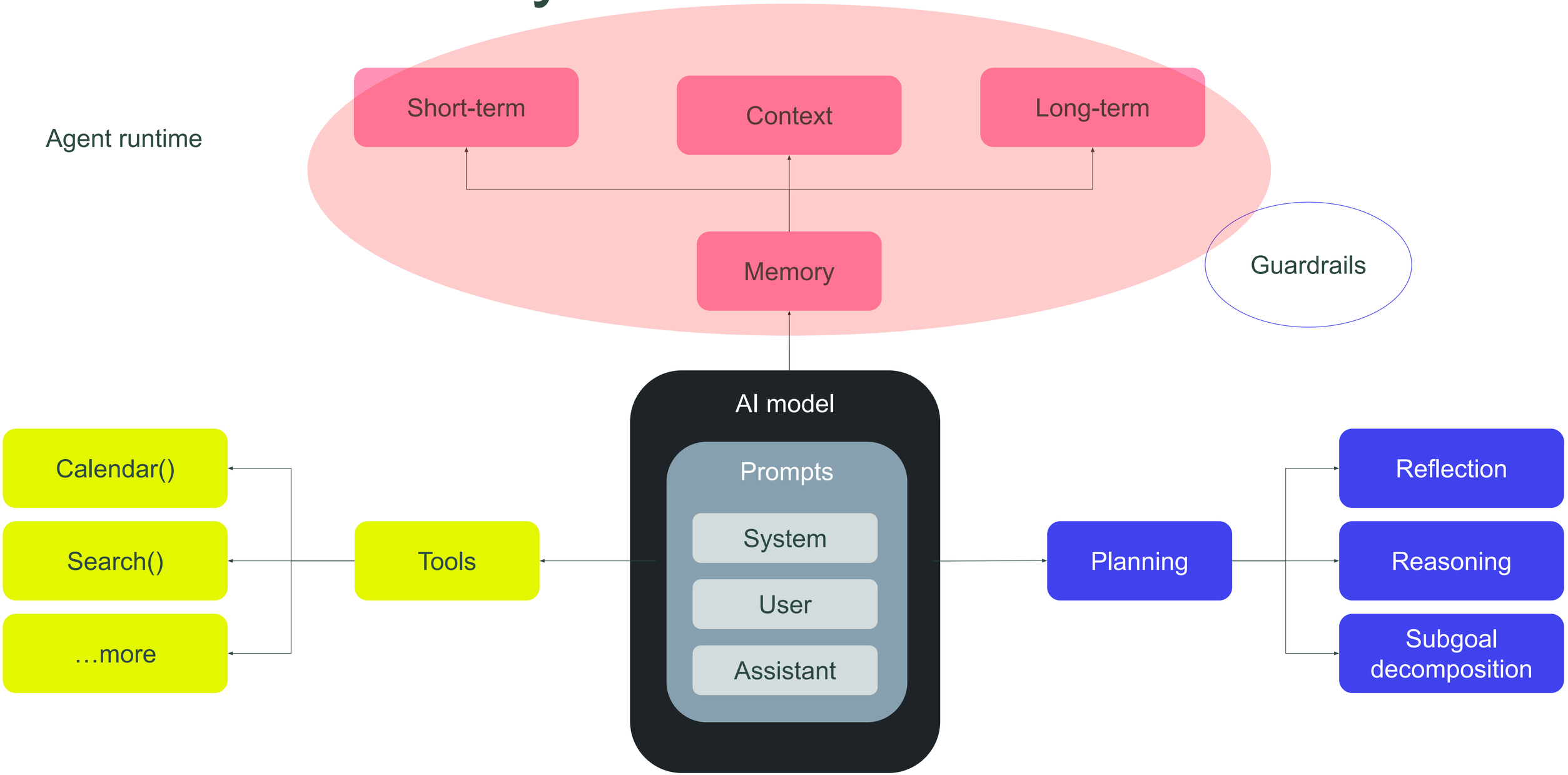
What We Already Know



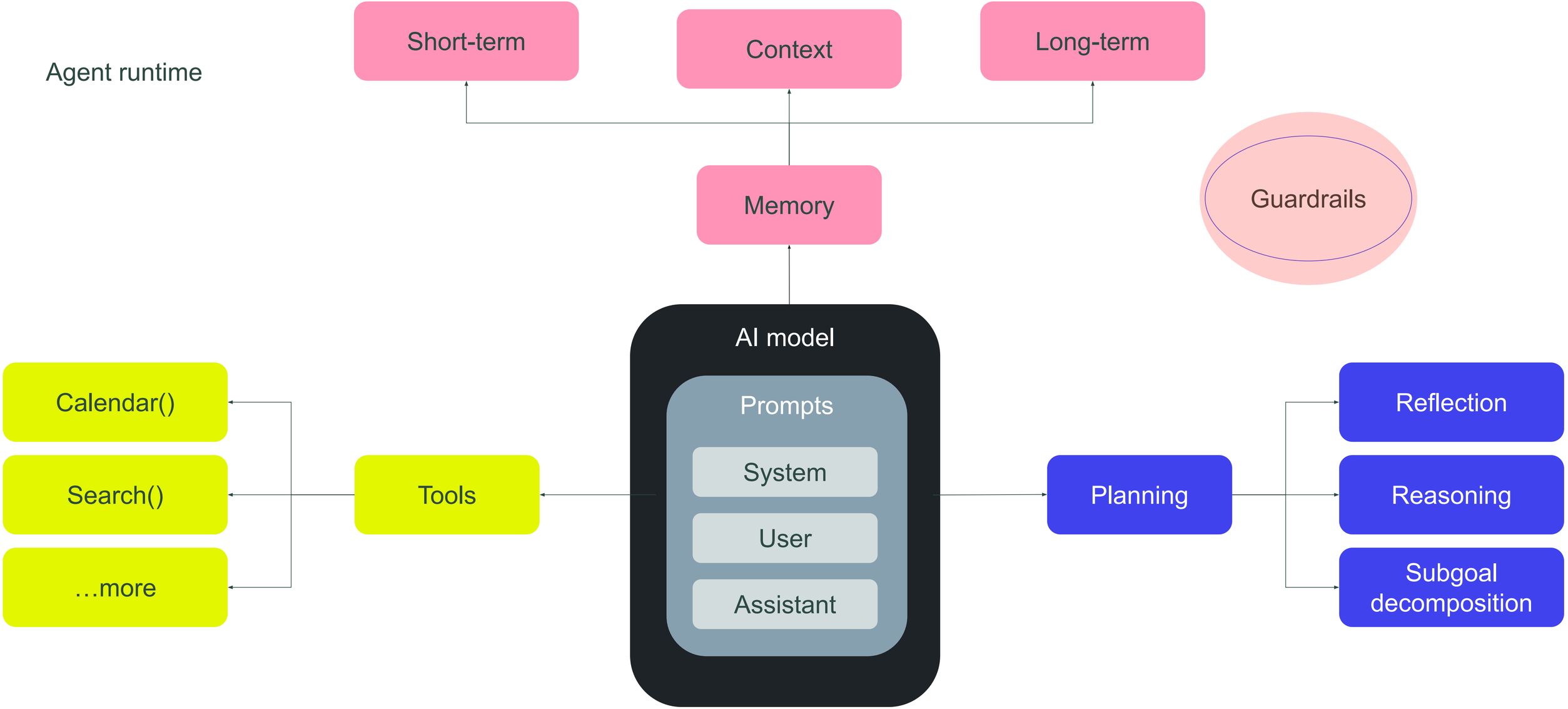
What We Already Know



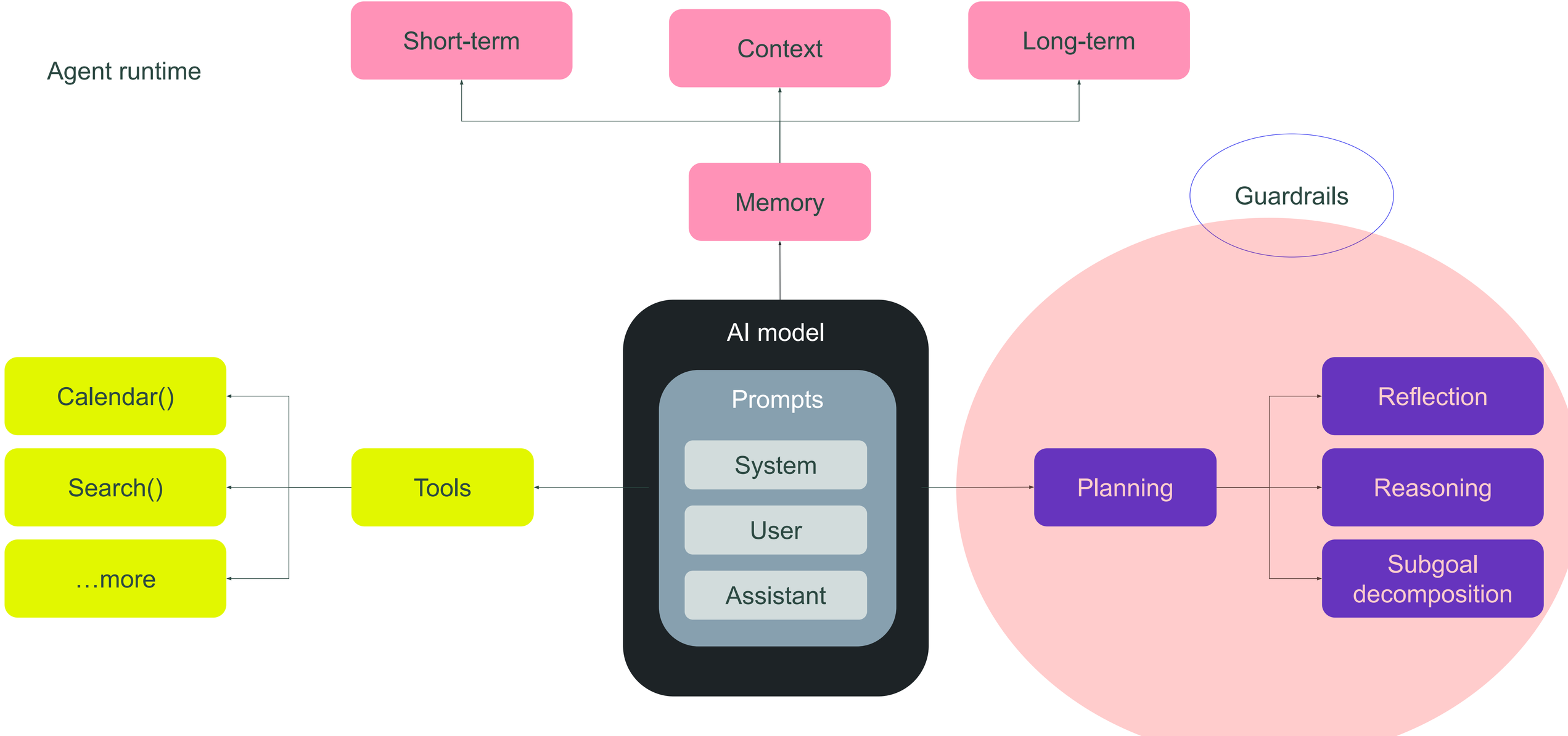
What We Already Know



What We Already Know

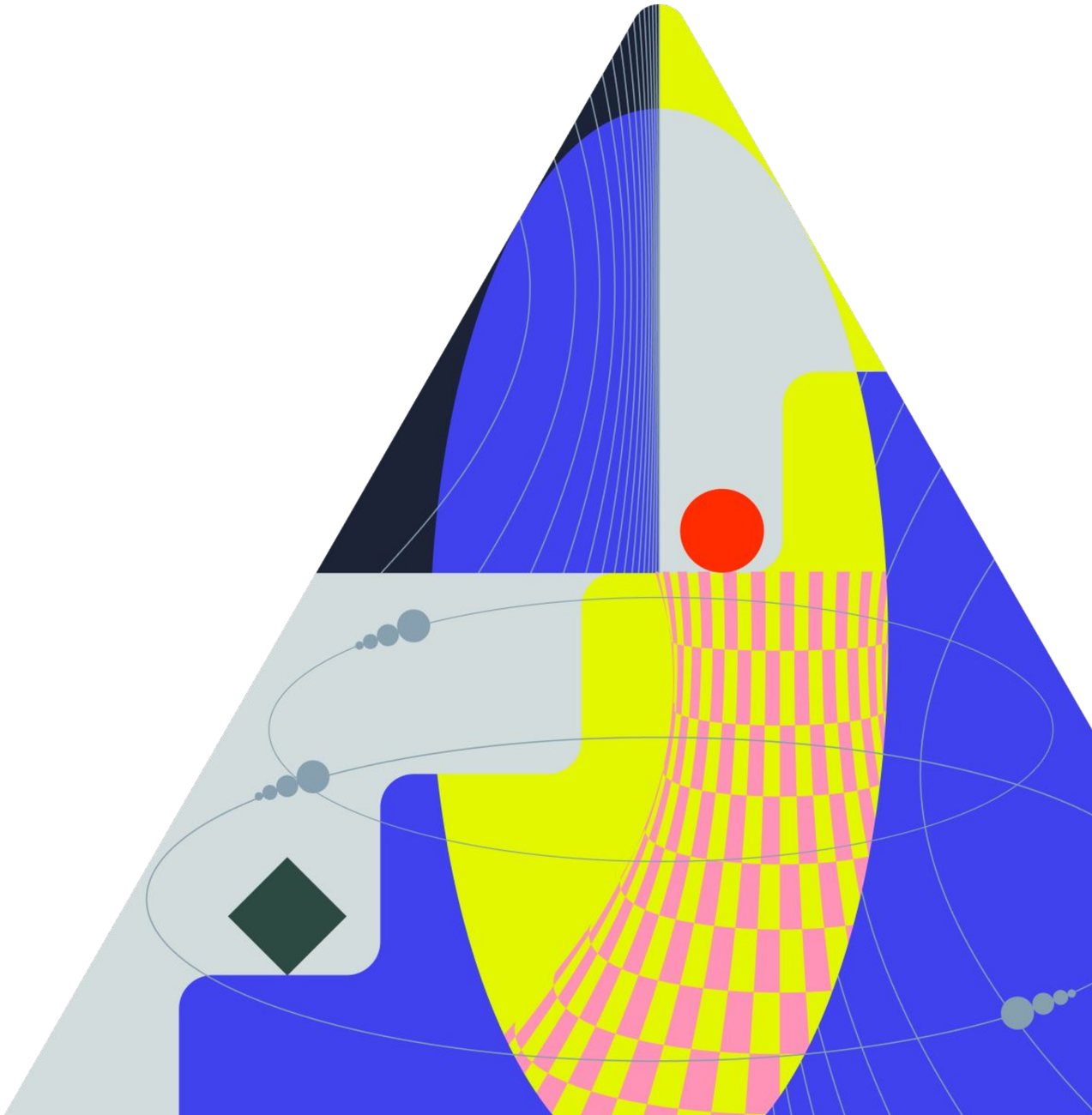


What We Already Know



Part 1

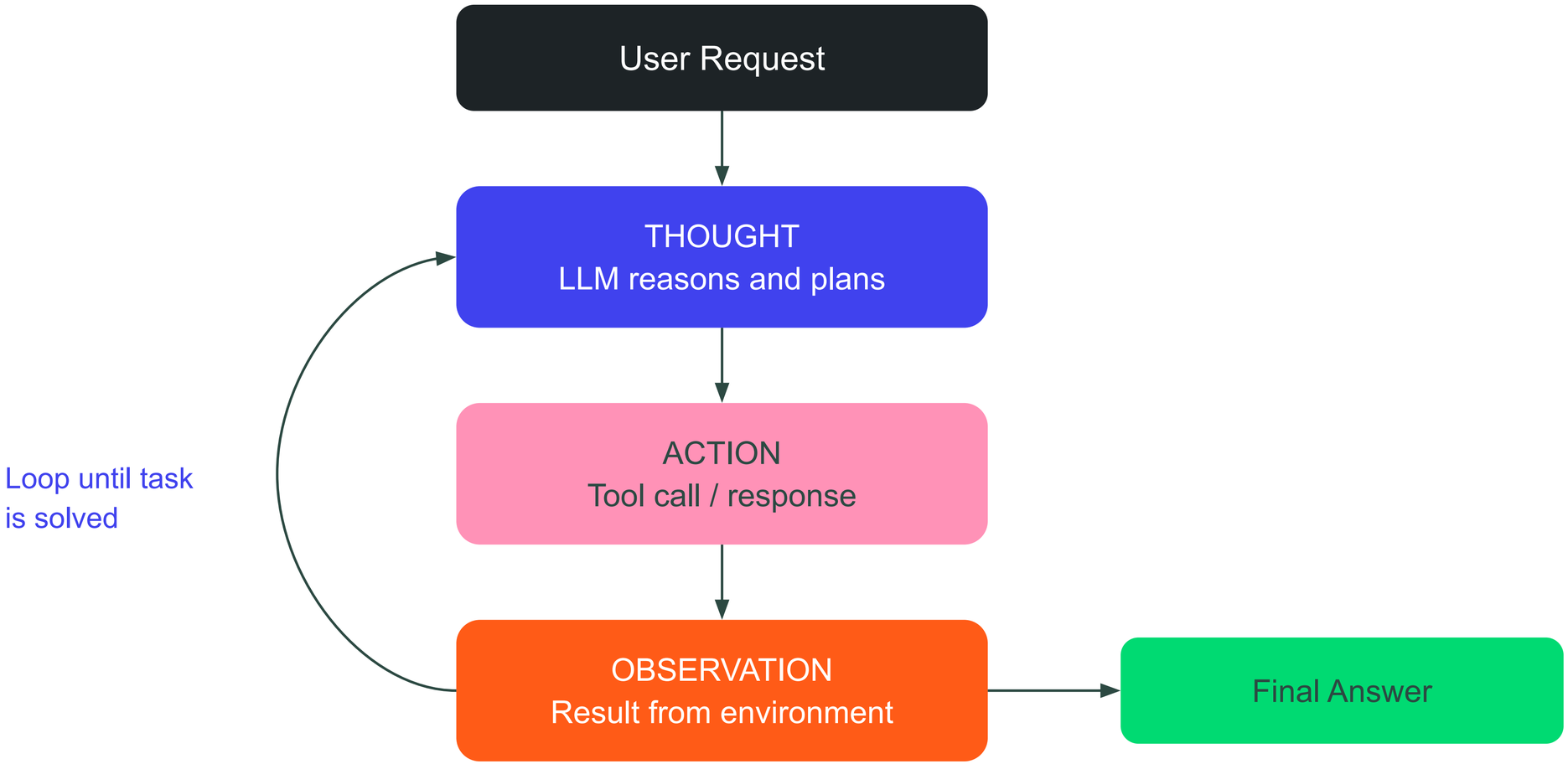
How Agents Think
TAO Loop



Thought → Action → Observation (TAO) Loop

The TAO cycle (Thought-Action-Observation) is the workflow of an artificial intelligence agent. It allows agents to dynamically adjust their responses, improve understanding, and improve the accuracy of the results

Thought → Action → Observation (TAO) Loop



THOUGHT: Agent's Internal Reasoning

THOUGHT Type	Example
Planning	"I'll break the task into 3 steps: find booking → check policies → update"
Analysis	"Based on the API error, the issue is in the date format"
Self-reflection	"My previous answer was too generic"

Four Reasoning Strategies

CoT (Chain-of-Thought)

Step-by-step reasoning before answering.
Single generation.
«Let's think step by step.»

ReAct

Alternating Think ↔ Act ↔ Observe.
Suitable for tool-use tasks

Reasoning (o1-style)

Built into the model during training
(o1, DeepSeek R1).
<think>...</think> — self-reflection.

Chain-of-Thought (CoT)

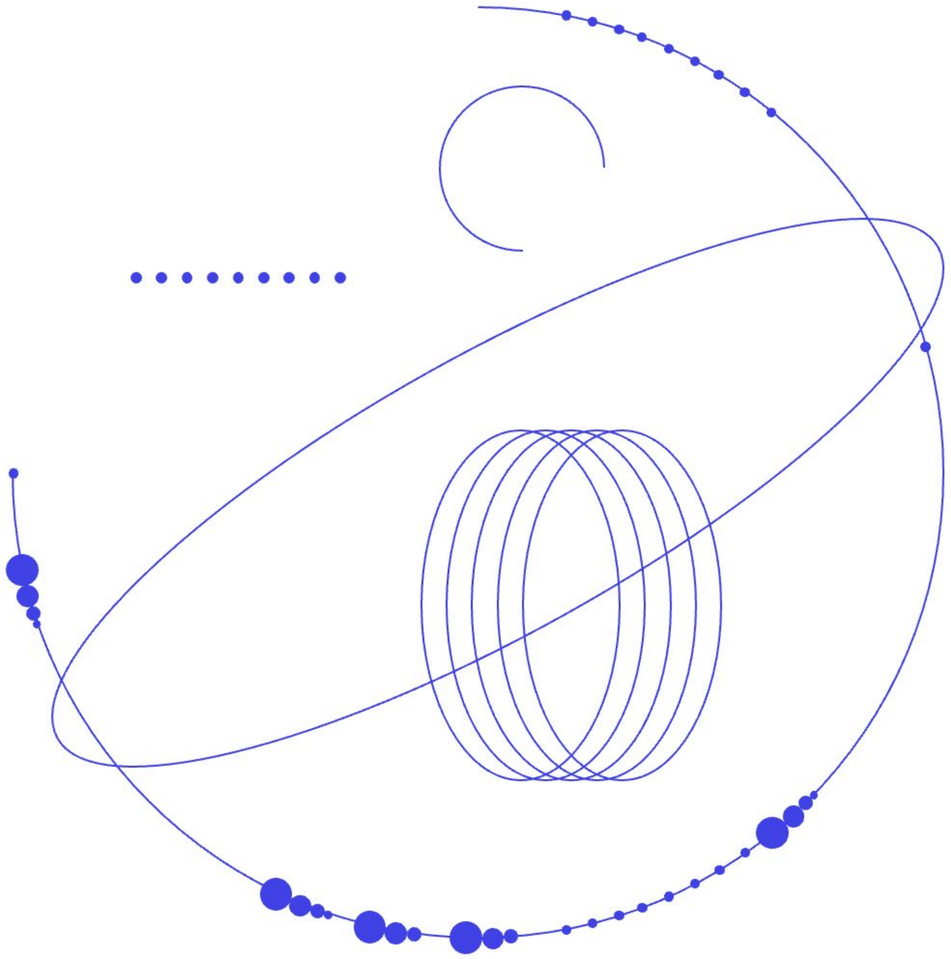
```
Question: What is 15% of 200?

Thought: Let's go step by step.
10% of 200 = 20
5% of 200 = 10
  15% = 20 + 10 = 30

Answer: 30
```

Step-by-step reasoning within a single generation

No tools — text-based reasoning only

Well suited for logical and mathematical tasks

Problem: error accumulation — if step 2 is wrong, steps 3+ are built on a flawed basis. No external feedback.



ReAct (Reasoning + Acting)

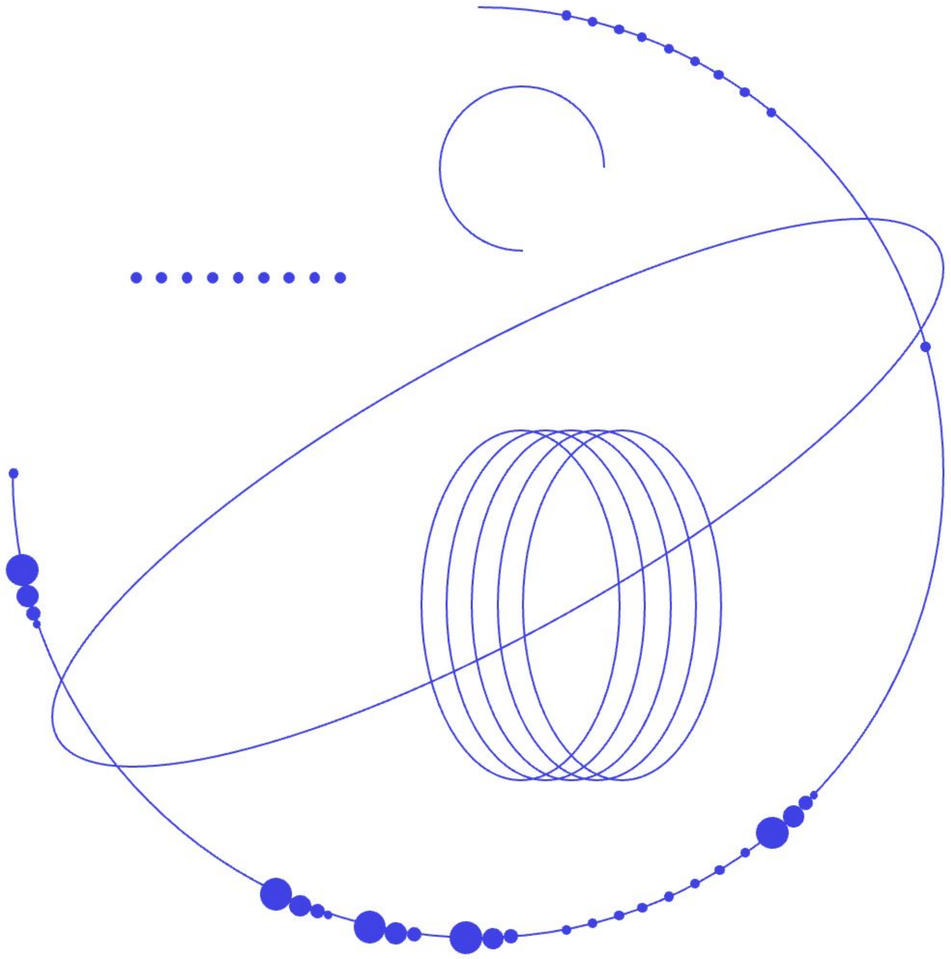
```
Thought: I need to check the weather in Paris.
Action:  search("weather in Paris")
Observation: 18°C, cloudy.

Thought: Now I know the weather, I can answer.
Action:  final_answer("It's 18°C and cloudy in
Paris.")
```

Alternating reasoning and actions

Tool usage

Ability to correct via Observation

ReAct > CoT for tool-based tasks

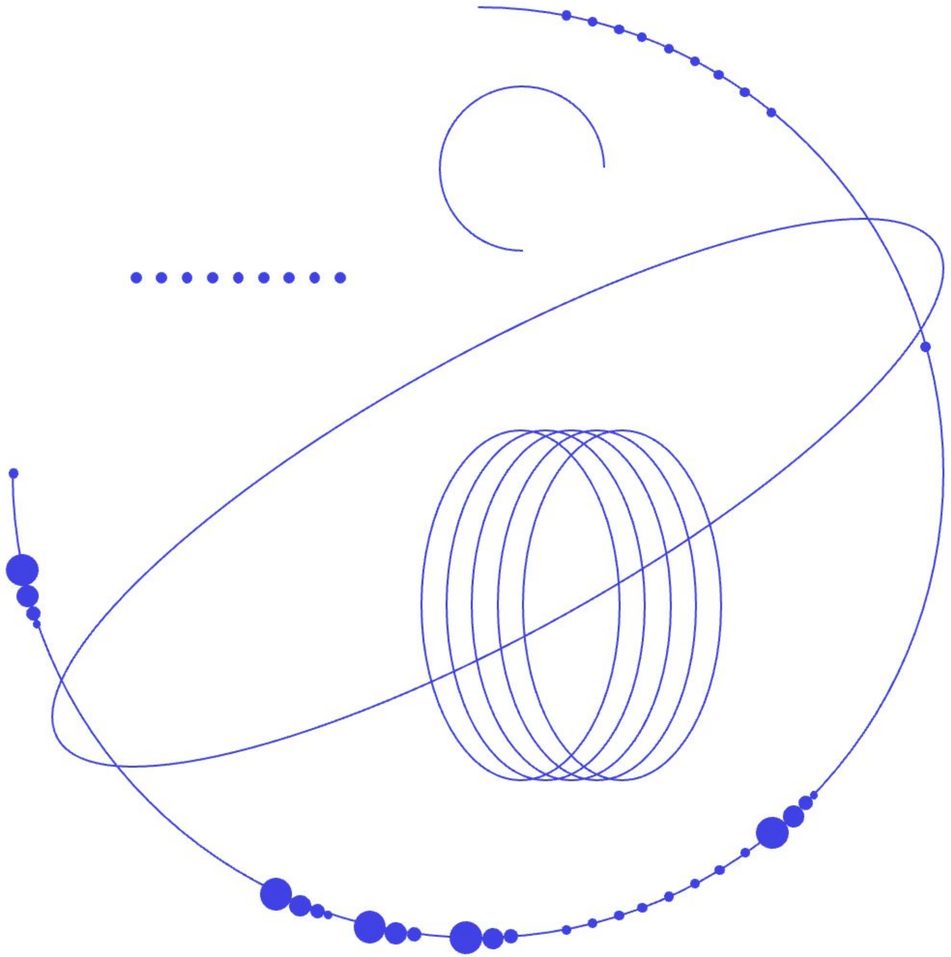


Reasoning models (o1-style)

Models like OpenAI o1, DeepSeek R1 — trained to think before answering.
Use special <think> and </think> tokens

```
<think>
Analyzing request... Need to consider the
same-class-only restriction...
Checking rebooking cost...
$40 for rebooking in economy...
</think>

Answer: Rebooking is available for $40 within economy
class.
```

Unlike CoT and ReAct: this is not a prompting strategy, but a model property built in during training



Visual Comparison of Three Strategies

CoT

Question
▼
Think step 1
▼
Think step 2
▼
Think step 3
▼
Answer

No tools
No correction
Single generation

ReAct

Question
▼
Think → Action
▼
Observe
▼
Think → Action
▼
Done? → Answer

With tools
With correction
Multiple generations

Reasoning (o1)

Question
▼
<think>
...hidden reasoning
...steps
</think>
▼
Answer

Built into model
Self-reflection
Single generation

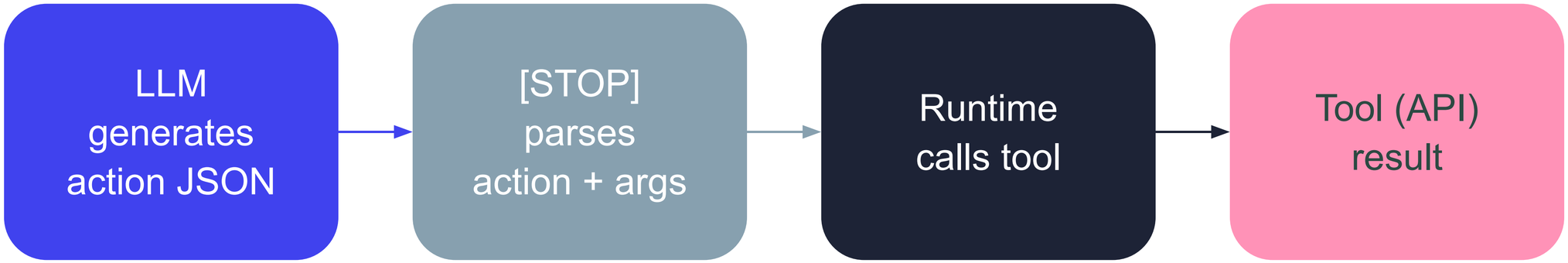
Strategy Comparison Table

Characteristic	CoT	ReAct	Reasoning (o1)
Step-by-step logic	Yes	Yes	Yes
External tools	No	Yes	Optional
Plan correction	No	Yes (Observation)	Internal
Where defined	Prompt	Prompt	Model training
Best for	Logic, math	Tool-based tasks	Complex planning
Cost	Low	Medium (N calls)	High (reasoning tokens)

ACTION + Tool Call Formats

Format	Description	Example
JSON Agent	Action as a JSON object	{"action": "search_flights", ...}
Function-calling	Native model format	Messages with tool_calls
Code Agent	Executable code generation	results = search_flights(...)

Stop and Parse



OBSERVATION: Feedback from the Environment

Observation Type	Example
API Data	{"flights": [...], "count": 3}
System Error	{"error": "rate_limit_exceeded", "retry_after": 30}
Confirmation	{"status": "booking_confirmed", "id": "BK-12345"}
User Input	"Yes, book this flight"

Three processing steps:

- 1. Parse — identify function and arguments
- 2. Execute — perform the action
- 3. Append — result → into agent context

Structured Output for Planning

```
from pydantic import BaseModel

class SubTask(BaseModel):
    agent_name: str
    description: str
    priority: int

class Plan(BaseModel):
    subtasks: list[SubTask]

structured_llm = llm.with_structured_output(Plan)
plan = structured_llm.invoke(
    "Break the rebooking task into subtasks"
)
# plan.subtasks → [SubTask(agent_name="flight", ...), ...]
```

Structured Output for Planning

```
from pydantic import BaseModel

class SubTask(BaseModel):
    agent_name: str
    description: str
    priority: int

class Plan(BaseModel):
    subtasks: list[SubTask]

structured_llm = llm.with_structured_output(Plan)
plan = structured_llm.invoke(
    "Break the rebooking task into subtasks"
)
# plan.subtasks → [SubTask(agent_name="flight", ...), ...]
```

Structured Output for Planning

```
from pydantic import BaseModel

class SubTask(BaseModel):
    agent_name: str
    description: str
    priority: int

class Plan(BaseModel):
    subtasks: list[SubTask]

structured_llm = llm.with_structured_output(Plan)
plan = structured_llm.invoke(
    "Break the rebooking task into subtasks"
)
# plan.subtasks → [SubTask(agent_name="flight", ...), ...]
```

Structured Output for Planning

```
from pydantic import BaseModel

class SubTask(BaseModel):
    agent_name: str
    description: str
    priority: int

class Plan(BaseModel):
    subtasks: list[SubTask]

structured_llm = llm.with_structured_output(Plan)
plan = structured_llm.invoke(
    "Break the rebooking task into subtasks"
)
# plan.subtasks → [SubTask(agent_name="flight", ...), ...]
```

Structured Output for Planning

```
from pydantic import BaseModel

class SubTask(BaseModel):
    agent_name: str
    description: str
    priority: int

class Plan(BaseModel):
    subtasks: list[SubTask]

structured_llm = llm.with_structured_output(Plan)
plan = structured_llm.invoke(
    "Break the rebooking task into subtasks"
)
# plan.subtasks → [SubTask(agent_name="flight", ...), ...]
```

TAO Loop in Action: Flight Rebooking

1
```
Thought: Find the booking
Action:  get_booking(user_id="U-1234")
Obs: {booking_id: "BK-789", flight: "SU2454"}
```

2
```
Thought: Searching flights for tomorrow
Action:  search_flights("SVO", "CDG", "04-16")
Obs: [{SU2456: 37k₽}, {AF1147: 42k₽}]
```

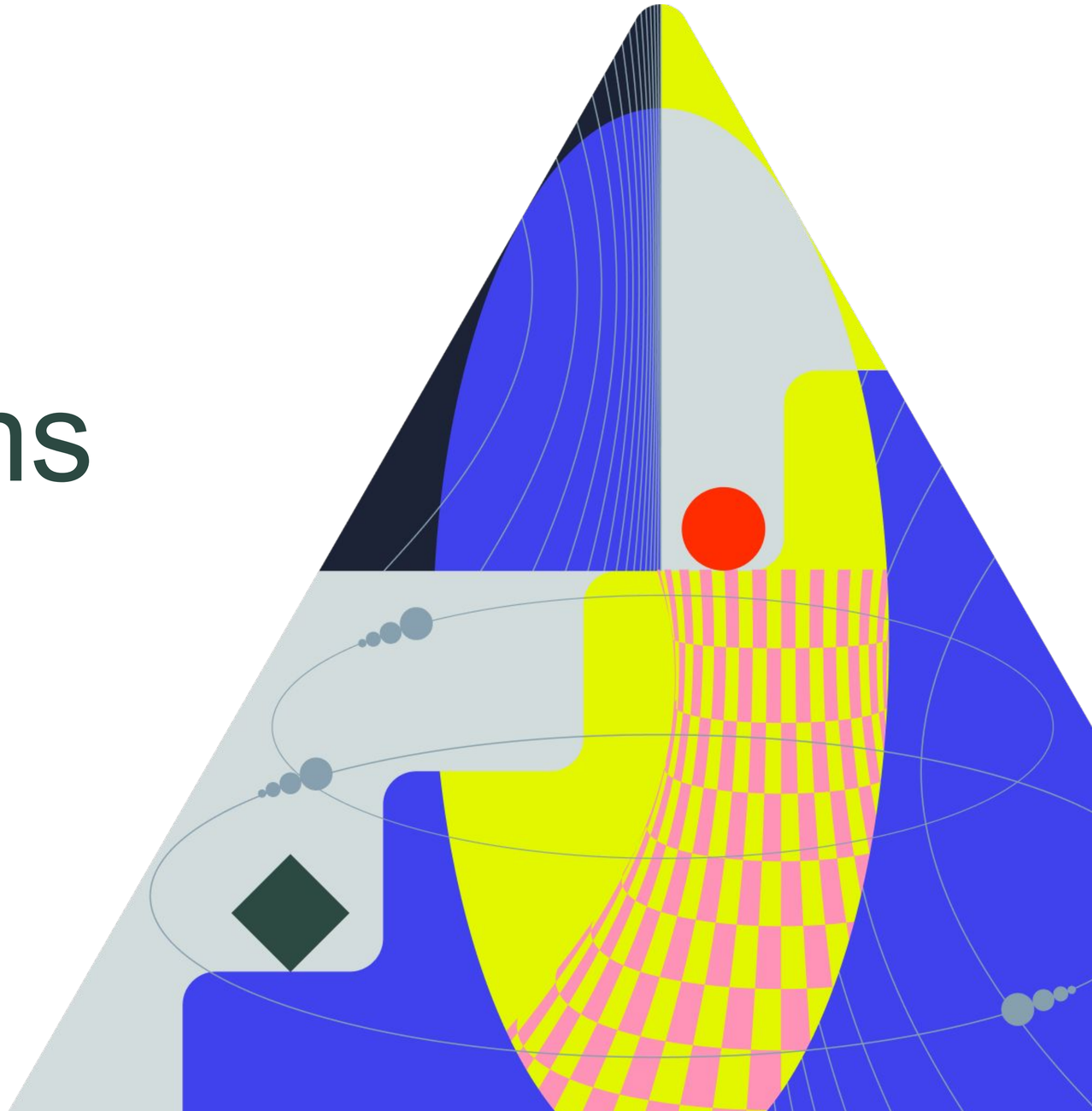
3
```
Thought: I'll check the policies
Action:  get_policy("rebooking")
Obs: {fee: 3000₽, same class only}
```

4
```
Thought: I'll offer options to the user
Action:  final_answer("SU2456 or AF1147, $40 surcharge")
```

4 TAO loop iterations, 3 tool calls, 1 final answer

Part 2

Multi-Agent Systems (MAS)



Why Do We Need MAS?

Problem — a single agent as task complexity grows:

Gets lost with too many tools

Mixes up subtask contexts

Difficult to debug

Becomes unreliable

Solution: specialized agents, each an expert in their domain

Single-Agent vs Multi-Agent

	Single-Agent	Multi-Agent
Strengths	Simplicity, no coordination, fewer resources	Complex tasks, specialization, parallelism, fault tolerance
Weaknesses	Limited by context, confused with many tools	Coordination overhead, debugging complexity
When to choose	Simple tasks (2–5 tools)	Many tools, diverse expertise

Key Advantages of MAS:

- More complete coverage of problem aspects
- Mutual verification and result validation
- Small specialized models instead of one large model
- Resilience to individual agent failures

6 Principles of MAS

DECOMPOSITION

Breaking the task
into subtasks

SPECIALIZATION

Each agent is
an expert in their domain

COMMUNICATION

Efficient exchange
of information

COORDINATION

Aligning actions
and aggregating results

VERIFICATION

Critics validate
intermediate results

FEEDBACK

Correction based on
info from other participants

Three Types of Inter-Agent Interaction

DEBATE

```
Agent A: "$450"
↔ arguments ↔
Agent B: "$382"
▼
Consensus: "$382.50"
```

When:

Accuracy needed,
fact-checking, ethics/risks

COLLABORATION

```
Agent 1: Flight →
Agent 2: Policy →
Agent 3: Booking →
▼
Coordinator aggregates
```

When:

Clear pipeline,
diverse skills, efficiency

COMPETITION

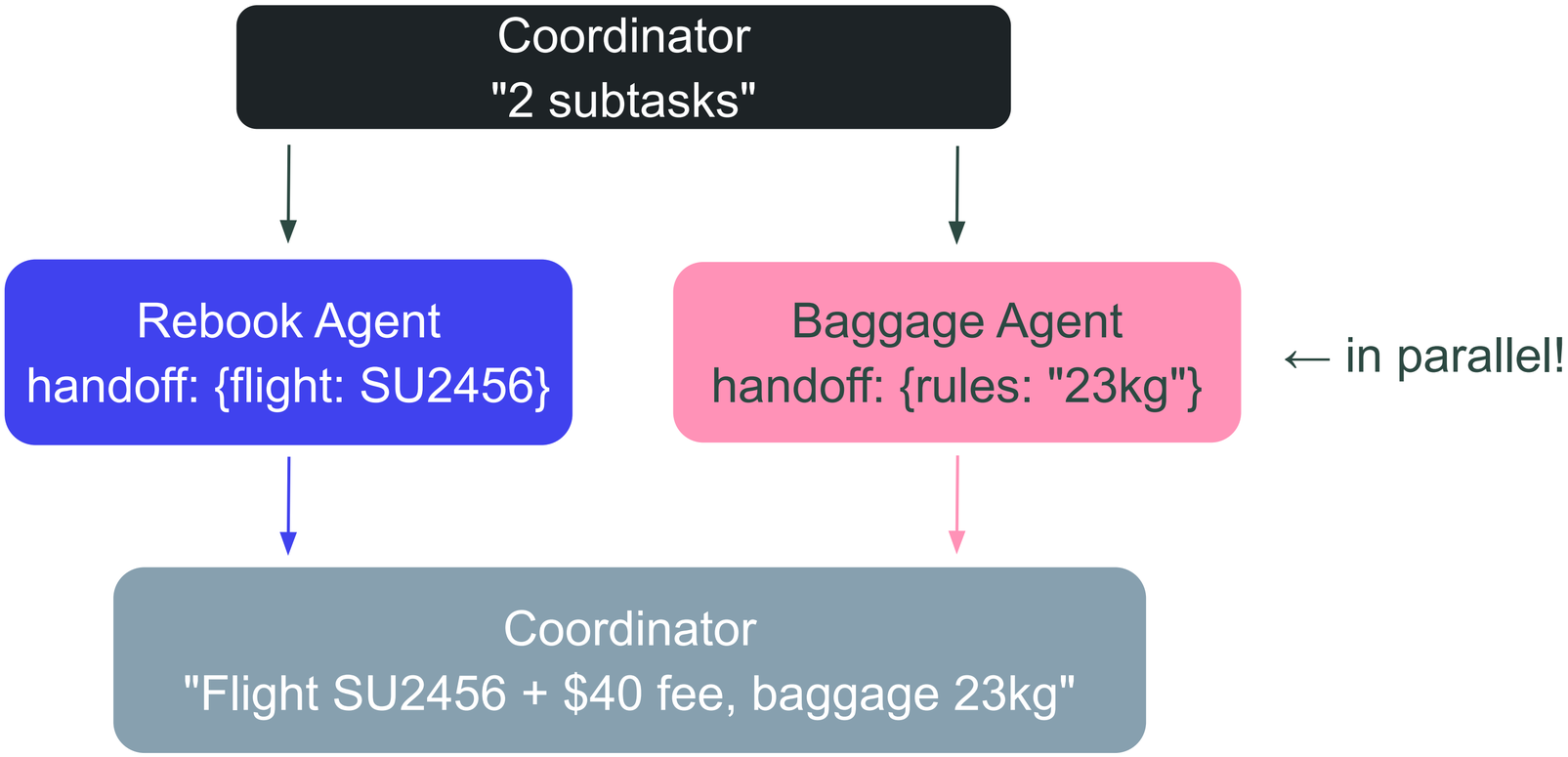
```
Agent A: "cheapest"
Agent B: "fastest"
Agent C: "comfort"
▼
Evaluator picks the best
```

When:

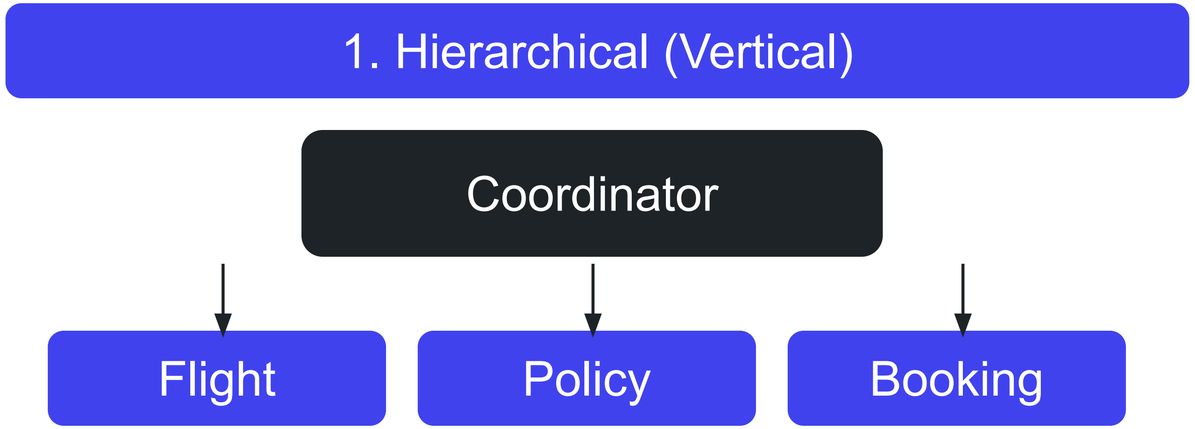
Multiple strategies,
creative tasks

Handoff — Key Collaboration Pattern

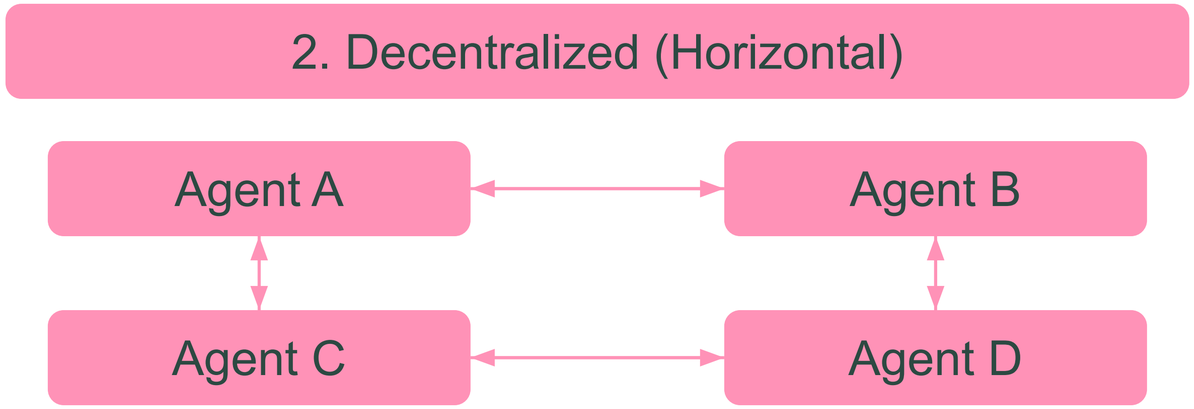
```
User: "I want to rebook and ask about baggage"
```



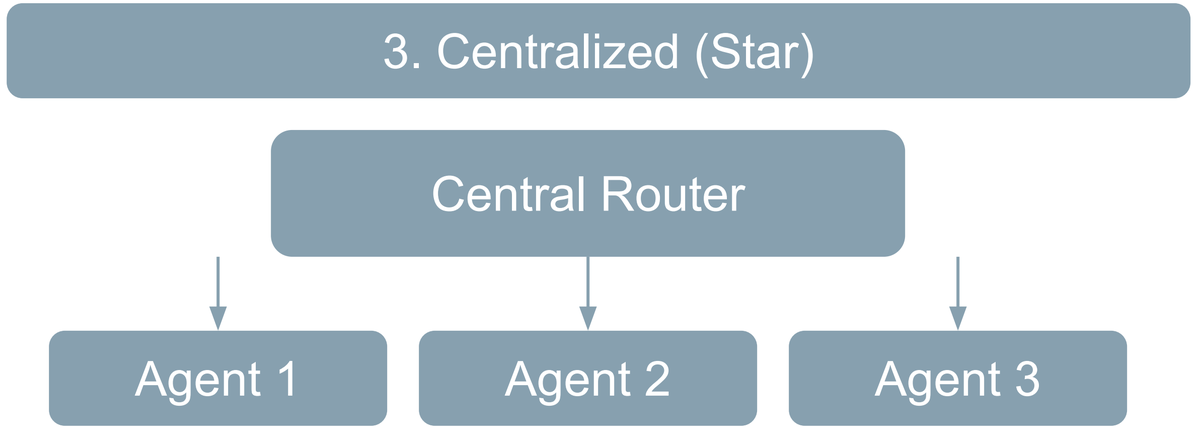
MAS Architectures



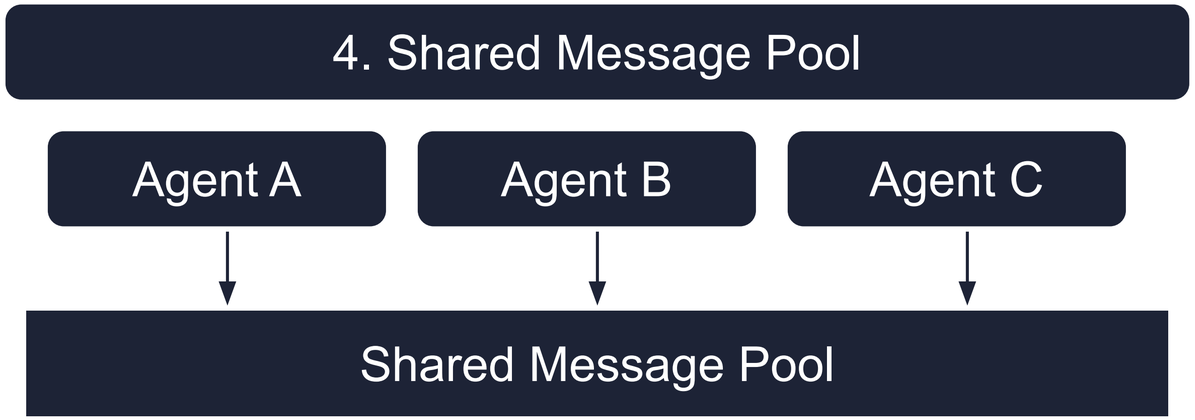
MAS Architectures



MAS Architectures



4 MAS Architectures



Architecture Comparison

Criterion	Hierarchical	Decentralized	Centralized	Shared Pool
Task complexity	High	Medium-High	Low-Medium	Medium
Fault tolerance	Medium	High ★	Low	Medium
Scalability	Medium	High ★	Low	High ★
Ease of debugging	High ★	Low	High ★	Medium
Predictability	High ★	Low	High ★	Medium

MAS Protocols + A2A

MCP (Model Context Protocol)

Agent ↔ Tool

Analogy: USB-C — a universal adapter for connecting AI to tools

Anthropic, Nov 2024

MAS Protocols + A2A

MCP (Model Context Protocol)

Agent ↔ Tool

Analogy: USB-C — a universal adapter for connecting AI to tools

Anthropic, Nov 2024

A2A (Agent-to-Agent)

Agent ↔ Agent

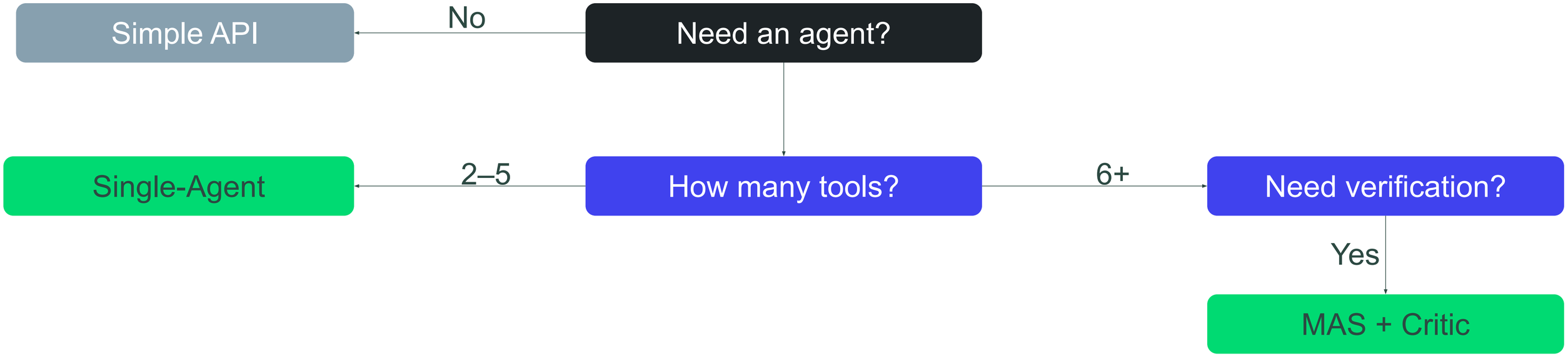
Analogy: Wi-Fi Direct — direct communication between devices

Google, Apr 2025

MAS Limitations

Category	Problem	Example
Resource Intensity	Each agent = LLM call. 3 agents × 5 iterations = 15+ calls	Cost, latency
Error Accumulation	One agent's error amplifies down the chain	A found the wrong flight → B booked it
Debugging	Hard to tell which agent failed in a long chain	Need tracing of every step

Decision Guide



What is the task structure?

Clear hierarchy → Hierarchical	Peer experts → Decentralized	Many subsystems → Centralized	Async processing → Shared Pool

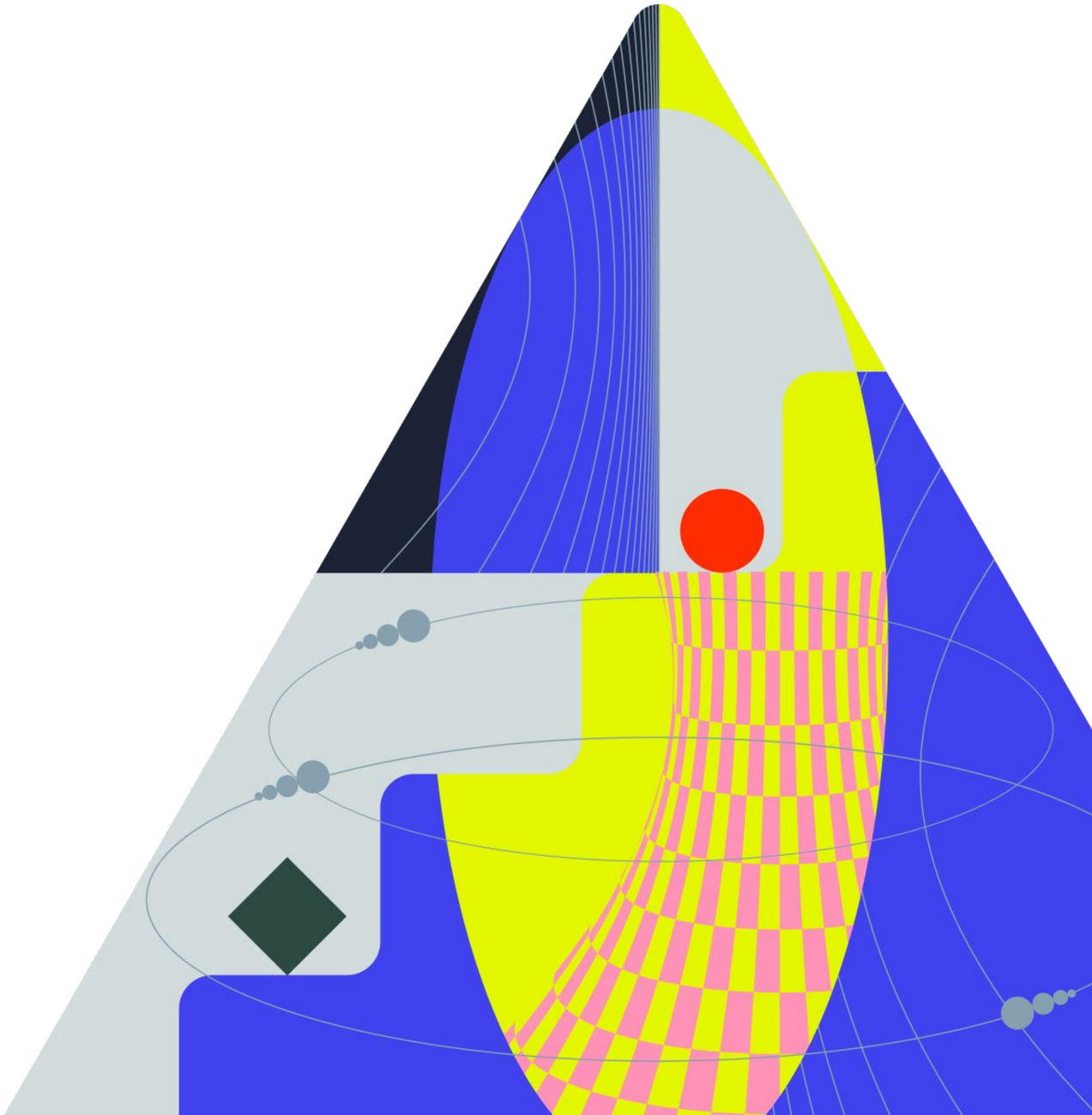
Anti-patterns — What NOT to Do

Anti-pattern	Problem	Solution
MAS for a simple task	Overhead > benefit	Start with single-agent
10+ agents at once	Impossible to debug	Incrementally: 2–3 → validate
No critic in critical domains	Errors reach the user	Always use Critic for critical data
All agents on expensive models	Expensive and slow	Small models for simple agents
No tracing	Impossible to debug	Log every TAO step

Rule: architecture is chosen to fit the task, not the other way around!

Part 3

Multimodal Agents

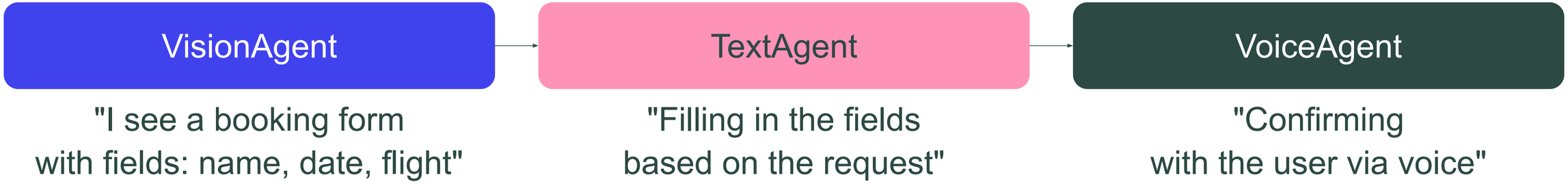


Multimodal Agents

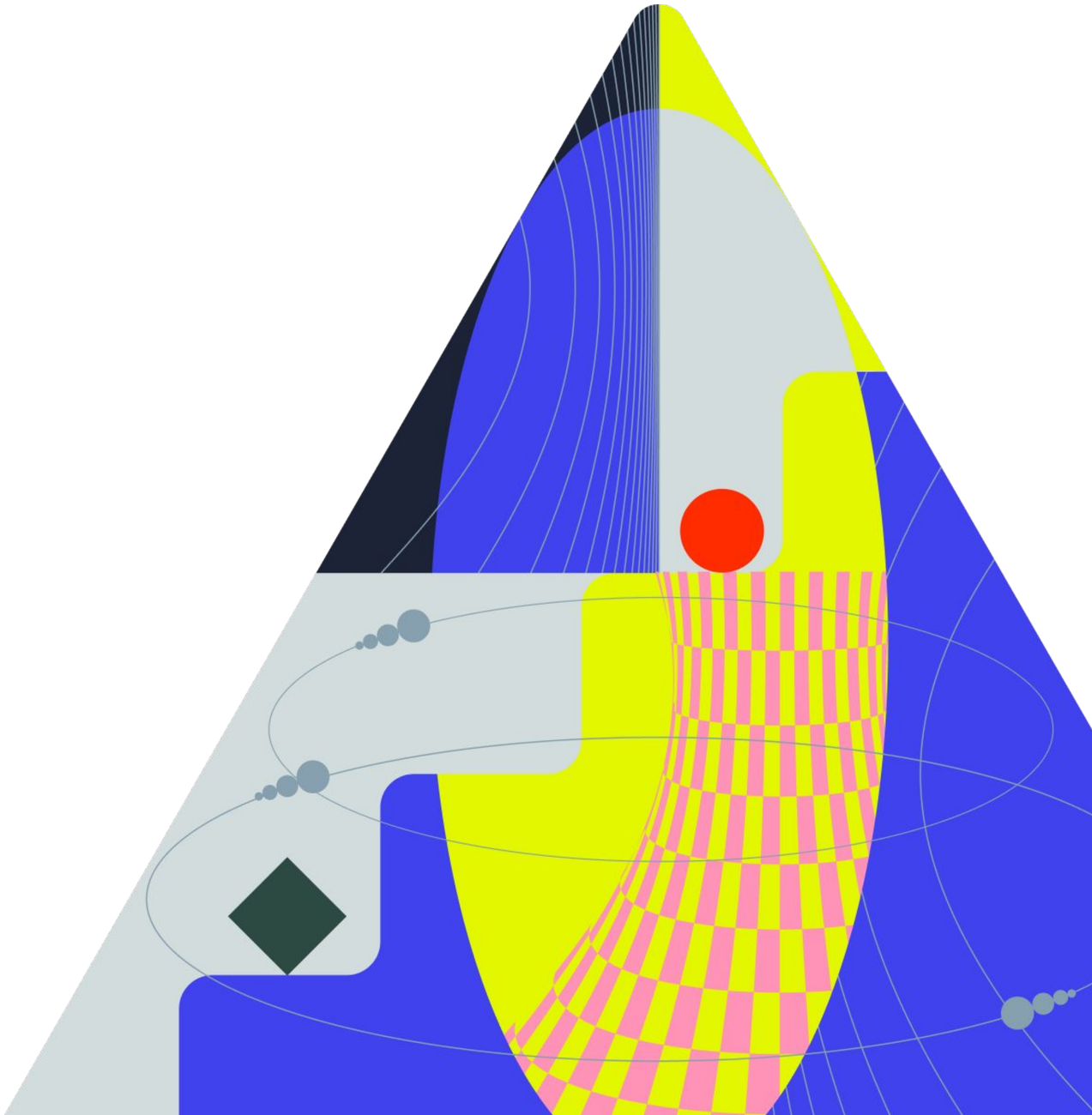
Type	Modality	Example Tasks
Vision	Images, Video stream	UI analysis, document OCR, visual QA
Voice	Audio	Voice assistants, call centers

Multimodal Agents

Multimodal MAS:



Summary



Conclusion

1. How a single model thinks

TAO loop, strategies: CoT, ReAct, Reasoning, Plan & Execute

2. How multiple models work as a team

MAS: architectures, interaction, protocols

3. How agents go beyond text

Multimodality: vision, voice, video

Key Takeaways:

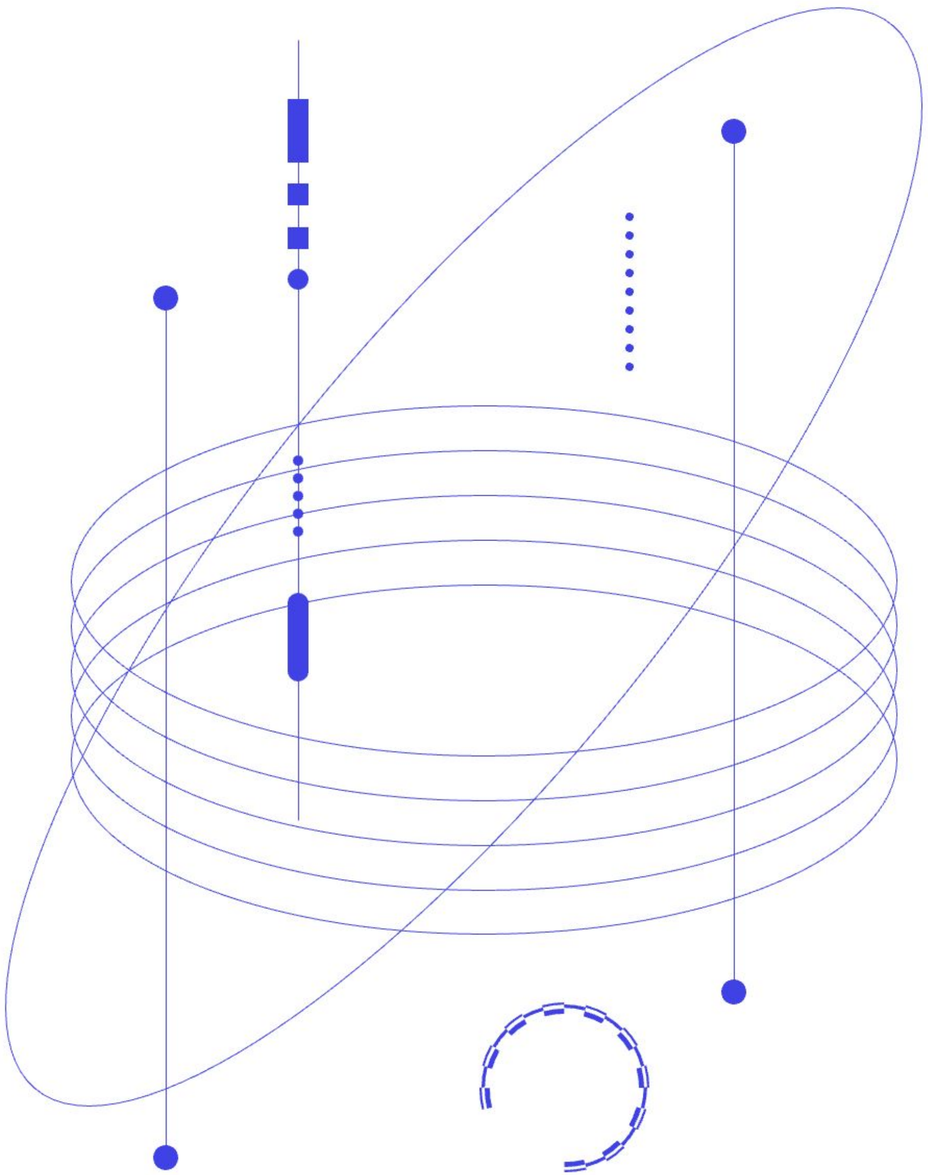
TAO loop — the fundamental pattern underlying every agent

ReAct > CoT for tool-based tasks

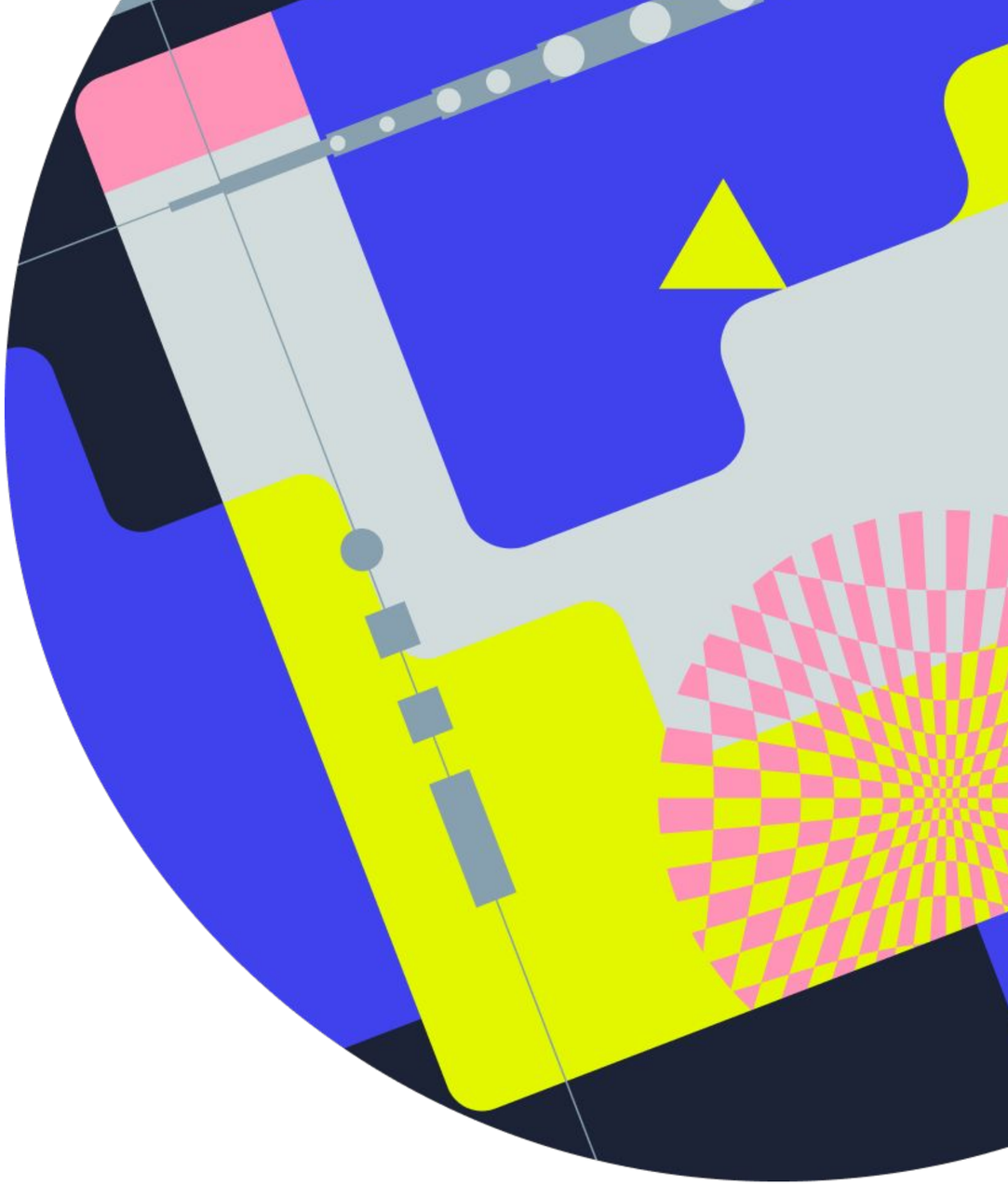
MAS solves specialization, verification, and scalability challenges

4 architectures — no "best" one, only the right fit for the task

Multimodality — the next frontier



Practice

Building a MAS for an airline:

1. Single Agent (ReAct) — reinforcing TAO
2. Hierarchical MAS — coordinator + specialists
3. Critic Agent — quality assurance
4. Benchmark — comparing architectures